

sensors



Article

Adaptive Grasp Pose Optimization for Robotic Arms Using Low-Cost Depth Sensors in Complex Environments

Aiguo Chen, Xuanfeng Li, Kerui Cen and Chitin Hon



<https://doi.org/10.3390/s25030909>

Article

Adaptive Grasp Pose Optimization for Robotic Arms Using Low-Cost Depth Sensors in Complex Environments

Aiguo Chen ^{1,2} , Xuanfeng Li ^{1,2} , Kerui Cen ^{1,2}  and Chitin Hon ^{1,2,*} 

¹ Faculty of Innovation Engineering, Macau University of Science and Technology, Taipa, Macau SAR 999078, China; holden_sh@126.com (A.C.); 3240005919@student.must.edu.mo (X.L.); 3240007122@student.must.edu.mo (K.C.)

² The Institute of Systems Engineering, Macau University of Science and Technology, Macau SAR 999078, China

* Correspondence: cthon@must.edu.mo; Tel.: +853-8897-2165

Abstract: This paper presents an efficient grasp pose estimation algorithm for robotic arm systems with a two-finger parallel gripper and a consumer-grade depth camera. Unlike traditional deep learning methods, which suffer from high data dependency and inefficiency with low-precision point clouds, the proposed approach uses ellipsoidal modeling to overcome these issues. The algorithm segments the target and then applies a three-stage optimization to refine the grasping path. Initial estimation fits an ellipsoid to determine principal axes, followed by nonlinear optimization for a six-degree-of-freedom grasp pose. Validation through simulations and experiments showed a target grasp success rate (TGSR) of over 83% under low noise, with only a 4.9% drop under high noise—representing a 68.0% and a 42.4% improvement over GPD and PointNetGPD, respectively. In real-world tests, success rates ranged from 95 to 100%, and the computational efficiency was improved by 56.3% compared to deep learning methods, proving its practicality for real-time applications. These results demonstrate stable and reliable grasping performance, even in noisy environments and with low-cost sensors.

Keywords: grasp pose estimation; ellipsoidal modeling; robotic arm systems; nonlinear optimization



Received: 14 November 2024

Revised: 29 January 2025

Accepted: 30 January 2025

Published: 3 February 2025

Citation: Chen, A.; Li, X.; Cen, K.; Hon, C. Adaptive Grasp Pose Optimization for Robotic Arms Using Low-Cost Depth Sensors in Complex Environments. *Sensors* **2025**, *25*, 909. <https://doi.org/10.3390/s25030909>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Dexterous manipulation of robots has broad applications in industries such as manufacturing, logistics, and healthcare. In industrial production, robots improve efficiency by performing tasks like sorting, packaging, and painting. In logistics systems, mobile robots and arms collaborate for intelligent de-stacking, picking, and sorting. In daily life, service robots assist with tasks such as serving tea or organizing, offering significant convenience.

Industrial robots based on artificial intelligence face significant challenges in efficiently grasping diverse objects [1–3], and while current grasping techniques are effective in specific scenarios, they often fail to meet the varied demands of industrial applications [4–6]. In both industrial and consumer-grade robot grasping tasks, the uncertainty in task execution can arise from several factors.

From a perception perspective, the grasping pose is derived from modeling and analyzing the scene data acquired by sensors. Whether using 2D or 3D data, strategies such as denoising, multi-frame accumulation, or improving sensor accuracy to reduce algorithmic strain may be employed to increase the success rate of task execution. However, these strategies often come with increased time, computational power, or equipment costs. To reduce these costs and make robotic arms more widely applicable, manufacturers have

introduced miniaturized, low-cost 3D sensors, such as Intel's RealSense and Microsoft's Kinect. However, the perception accuracy of these devices is constrained by the cost, achieving only centimeter-level (5–10 cm) measurement precision. This has facilitated the widespread adoption of low-cost robots; however, the limited perception accuracy has resulted in a lack of robust algorithms for object grasping with these sensors. In fact, the trade-off between device cost, computational cost, and task execution robustness has been a central theme in the development of the robotics community. This paper focuses on the uncertainty in robotic arm grasping tasks caused by imprecise and insufficient perception, and aims to develop a cost-effective, efficient, and robust grasping algorithm, which is key to broader applications [7,8].

In terms of algorithmic principles, grasping solutions can be categorized into model-based and model-free approaches [9]. The former involves high maintenance costs [10,11], while the latter is more flexible but faces challenges in generalization [12,13]. Parallel computing has facilitated the application of deep learning in grasp detection [14], where the use of RGB-D sensors in conjunction with neural networks such as CNN and PointNet has enhanced the accuracy of grasp prediction [15–18]. However, methods like GPD and PointNet face high data dependency and computational demands, limiting their industrial applications [19–23]. Current methods achieve success rates of 75–95% under controlled conditions; however, this rate is insufficient for dealing with clutter and occlusion in real-world scenarios [7,19,24–26]. Improving the robustness and adaptability of algorithms remains a key challenge [27–29].

This paper proposes a robust, efficient grasp pose estimation algorithm for a robotic arm with a two-finger gripper and consumer-grade depth camera. Key contributions include the following:

1. A PCA-based point cloud processing method for diverse target types and orientations, offering superior generalization.
2. A grasp strategy considering both target pose and environment for successful, collision-free grasps in complex scenes.
3. Millisecond-level grasp estimation using low-cost depth sensors, ensuring deployability with minimal size, weight, and power (SwaP) requirements.

2. Methods

2.1. System Overview

The hardware and algorithm framework are shown in Figure 1. The system uses an eye-in-hand setup with a RealSense D455 (RealSense D455 camera, Intel Corporation, Santa Clara, CA, USA 2020.) camera on an Elfin5 (Elfin5, JAKA Robotics, Shenzhen, China, 2021.) robotic arm, effective within a 0.5–2 m range for small to medium grasping tasks.

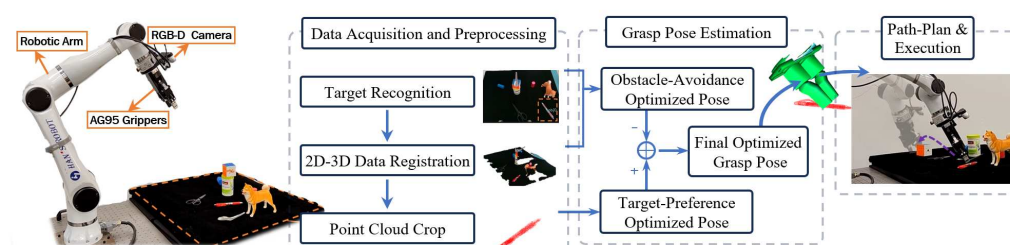


Figure 1. System algorithm flowchart.

Grasping begins with the accurate localization of the target using MobileSAM for pixel-wise segmentation, followed by point cloud alignment. The system then separates the environment and target point clouds. A multi-objective optimization approach estimates

the optimal grasp pose by maximizing grasp success while avoiding collisions, as further explained in Sections 2.2 and 2.3.

2.2. Data Acquisition and Preprocessing

In this study, we adopt MobileSAM [30], a lightweight image segmentation network optimized for mobile devices. MobileSAM uses a convolutional neural network to extract target regions from images and integrates information across different scales to accurately segment the targets. Compared to traditional image segmentation methods, this approach significantly reduces computational costs while maintaining accuracy, making it suitable for resource-constrained devices. The model processes normalized and resized images to produce segmentation results, followed by post-processing to generate the final mask. Test results are shown in Figure 2.

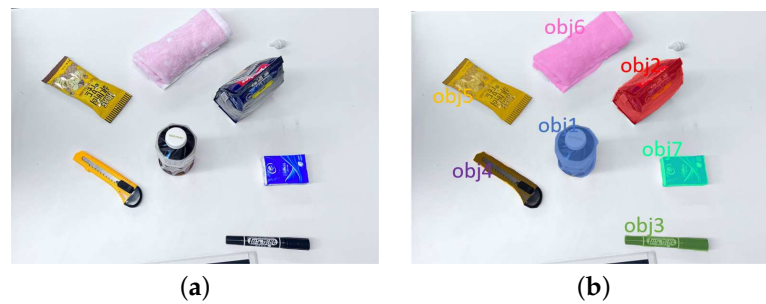


Figure 2. Target localization and segmentation: (a) original image; (b) segmentation result.

Pixel alignment between the RGB camera and the depth camera is a critical step in achieving multimodal data fusion. Camera calibration provides the intrinsic and extrinsic matrices for both the RGB and depth cameras. Specifically, a calibration board is used to calibrate the two cameras, obtaining the intrinsic matrices K_{RGB} and K_{Depth} for each camera, as well as the extrinsic matrices R and T between them. The pixel coordinates (u_d, v_d) from the depth image are mapped to 3D space coordinates $(\hat{X}, Y, Z)$, which are then transformed into the RGB camera coordinate system using the extrinsic matrices. Finally, these 3D coordinates are projected back into the RGB pixel coordinate system (u_r, v_r) using the intrinsic matrix of the RGB camera. This mapping relationship can be expressed by the following equations:

$$\begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = K_{RGB} \cdot \left(R \cdot \begin{bmatrix} \hat{X} \\ Y \\ Z \end{bmatrix} + T \right) \quad (1)$$

Through the aforementioned process, the depth image can be reprojected into the coordinate system of the RGB image, achieving pixel-level alignment. Point cloud processing is a crucial step in analyzing and manipulating 3D point data obtained from depth images or 3D scanning devices. The first step involves generating a 3D point cloud using camera calibration parameters and the depth image. Specifically, the 3D coordinates (X, Y, Z) in the camera coordinate system are calculated for each pixel (u, v) in the depth image along with its corresponding depth value d . This process can be achieved using the following equations:

$$X = (u - u_0) \cdot d / f_x \quad (2)$$

$$Y = (v - v_0) \cdot d / f_y \quad (3)$$

$$Z = d \quad (4)$$

where f_x and f_y are the focal lengths of the camera, and (u_0, v_0) are the coordinates of the principal point.

After the above process, we can align the results from 2D image recognition with the 3D sensor data on a pixel-by-pixel basis to obtain the 3D information of the target object. Even in cases where the target overlaps with the background, the object can still be separated from the environment. However, in more complex scenarios, relying solely on instance segmentation from 2D images may not be robust enough. For example, when the object and background objects have similar colors or textures, and there is significant foreground-background overlap, the instance segmentation results on the image may suffer from boundary overflow, which could affect the extraction of the 3D information of the target. To address this, we employ a clustering strategy to exclude point clouds that do not belong to the target. The implementation principle is described below: We introduce the Euclidean clustering method, which effectively removes misidentified objects in the background under overlapping environments by considering the spatial distance information of objects in the point cloud. Even if instance segmentation confuses background objects with foreground objects, their positions in three-dimensional space typically differ significantly. Euclidean clustering calculates the Euclidean distance between points and groups closely located points into the same cluster, thereby effectively separating foreground and background objects. For each point, Euclidean clustering computes its neighborhood and assigns these points to the same cluster based on a preset distance threshold ϵ . If a background object is spatially far from a foreground object, even if it is mistakenly segmented as foreground in the 2D image, Euclidean clustering can exclude it from the foreground. After clustering, background objects, due to their greater distance from foreground objects, are typically assigned to a separate cluster or labeled as noise, thereby being discarded. Ultimately, only the point cloud close to the foreground is retained, ensuring the effective removal of background objects.

It is important to note that if the target we wish to extract is largely occluded by an unrelated object and becomes invisible, this will first pose a challenge to instance segmentation. Thus, this strategy is primarily designed for optimizing the segmentation of overlapping foreground objects. Regarding overlapping background objects, we may need to consider how to circumvent and grasp targets in a more reasonable posture, which will be discussed in later in Section 2.3.

2.3. Grasp Pose Estimation

This section outlines the method for estimating the grasp pose, as introduced in Section 2.2. The coordinate system of the robotic arm is illustrated in Figure 3. The prismatic axis defines the gripper's movement, the tilt axis indicates the pitch angle, and the advance axis, derived from the cross product of the prismatic and tilt axes, defines the direction of gripper extension.

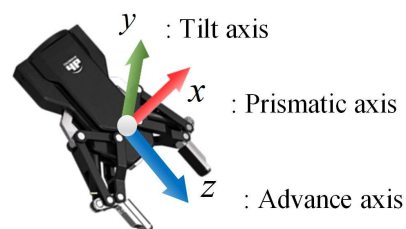


Figure 3. Two-finger parallel gripper coordinate system.

2.3.1. Ellipsoid Modeling

Grasp pose estimation aims to improve success by analyzing grasp factors. Objects are grasped by their thinnest part, modeled within an ellipsoid bounding box, as shown in Figure 4.



Figure 4. Grasping scene modeling: (a) point cloud of the grasping scene with registered and fused color images; (b) ellipsoid modeling of the grasping scene.

The ellipsoid model can be represented by the following general quadratic surface equation:

$$Ax^2 + By^2 + Cz^2 + 2Dxy + 2Exz + 2Fyz + 2Gx + 2Hy + 2Iz + J = 0 \quad (5)$$

Let the point cloud data be denoted as $P = \{(x_i, y_i, z_i)\}_{i=1}^N$. To apply the least squares method, a design matrix $\mathbf{A}$ and an observation vector $\mathbf{b}$ are constructed as follows:

$$\mathbf{A} = \begin{bmatrix} x_1^2 & y_1^2 & z_1^2 & 2x_1y_1 & 2x_1z_1 & 2y_1z_1 & 2x_1 & 2y_1 & 2z_1 \\ x_2^2 & y_2^2 & z_2^2 & 2x_2y_2 & 2x_2z_2 & 2y_2z_2 & 2x_2 & 2y_2 & 2z_2 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_N^2 & y_N^2 & z_N^2 & 2x_Ny_N & 2x_Nz_N & 2y_Nz_N & 2x_N & 2y_N & 2z_N \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} -1 \\ -1 \\ \vdots \\ -1 \end{bmatrix} \quad (6)$$

Solving the linear system $\mathbf{AX} = \mathbf{b}$ using the least squares method yields the parameter vector $\mathbf{X} = [A, B, C, D, E, F, G, H, I, J]^T$. These parameters define the general equation of the ellipsoid. The next step is to derive the ellipsoid's geometric parameters, such as the center position and the lengths of its axes.

By completing the square for the ellipsoid equation, the coordinates of the center point $Q_t = (x_0, y_0, z_0)$ can be calculated. Assuming the ellipsoid equation is in the standard form, as follows:

$$A(x - x_0)^2 + B(y - y_0)^2 + C(z - z_0)^2 + \dots = 1 \quad (7)$$

If the cross-product terms (i.e., the terms involving xy , xz , and yz , such as $2Dxy$, $2Exz$, $2Fyz$) are non-zero, it indicates that the ellipsoid is rotated relative to the coordinate axes. Specifically, the non-zero cross-product terms imply that the principal axes of the ellipsoid are not aligned with the coordinate axes but are instead rotated. In this case, a rotation matrix can be derived to transform the ellipsoid into its canonical or standard form, where the axes are aligned with the coordinate system.

The center coordinates of the ellipsoid are then given by

$$x_0 = -\frac{G}{A}, \quad y_0 = -\frac{H}{B}, \quad z_0 = -\frac{I}{C} \quad (8)$$

Next, the quadratic form matrix $\mathbf{Q}$ is constructed as follows:

$$\mathbf{Q} = \begin{bmatrix} A & D & E \\ D & B & F \\ E & F & C \end{bmatrix} \quad (9)$$

The eigenvalue decomposition of $\mathbf{Q}$ is performed as follows:

$$\mathbf{Q} = \mathbf{R}\mathbf{\Lambda}\mathbf{R}^T \quad (10)$$

where $\mathbf{\Lambda}$ is the diagonal matrix, with its diagonal elements being the eigenvalues $\lambda_1, \lambda_2, \lambda_3$, and the square roots of the reciprocals of the eigenvalues provide the lengths of the semi-axes of the ellipsoid, denoted as a, b, c :

$$a = \frac{1}{\sqrt{\lambda_1}}, \quad b = \frac{1}{\sqrt{\lambda_2}}, \quad c = \frac{1}{\sqrt{\lambda_3}} \quad (11)$$

where $\lambda_1 < \lambda_2 < \lambda_3$.

The transformation matrix from the ellipsoid coordinate system to the world coordinate system, $\mathbf{T}_{wt}$, includes both rotation and translation components:

$$\mathbf{T}_{wt} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ 0 & 1 \end{bmatrix} \quad (12)$$

where $\mathbf{R}$ is the rotation matrix composed of the eigenvectors, and $\mathbf{t}$ is the translation vector, representing the center position of the ellipsoid (x_0, y_0, z_0) .

$$(x_0, y_0, z_0) = \left(-\frac{G}{A}, -\frac{H}{B}, -\frac{I}{C} \right) \quad (13)$$

The lengths of the axes are

$$a = \frac{1}{\sqrt{\lambda_1}}, \quad b = \frac{1}{\sqrt{\lambda_2}}, \quad c = \frac{1}{\sqrt{\lambda_3}} \quad (14)$$

The complete transformation matrix is

$$\mathbf{T}_{wt} = \begin{bmatrix} \mathbf{R} & [x_0, y_0, z_0]^T \\ 0 & 1 \end{bmatrix} \quad (15)$$

These geometric parameters, including the rotation matrix and axis lengths, are depicted in Figure 5.

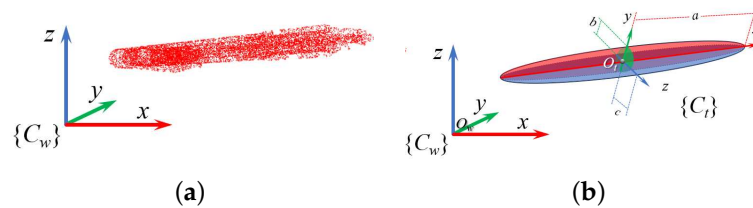


Figure 5. Ellipsoidal modeling of grasping target: (a) point cloud representation; (b) transformation matrix and axis lengths.

Based on the previous analysis, we can define the three-axis coordinate system of the modeled ellipsoid, with the transformation matrix from the world coordinate system to this new system being denoted as $\mathbf{T}_{wt}$. The rotation matrix $R_{wt} = [R_x, R_y, R_z]$, where R_x and R_z correspond to the longest and shortest axes of the ellipsoid $\{C_t\}$, respectively. The initial grasp pose is configured such that the marching axis aligns with the negative direction of the y -axis of $\{C_t\}$, and the prismatic axis aligns with the z -axis. This configuration ensures that the gripper can effectively grasp larger objects within its limited finger opening and closing

range. The tilt axis aligns with the x -axis, minimizing the risk of contact between the robotic arm and the target or surrounding objects. Consequently, the grasp pose transformation involves a ninety-degree rotation of the coordinate system around the x -axis, centered at the ellipsoid's center, aligning the transformed y -axis with the original z -axis. The resulting coordinate vector is then multiplied by a rotation matrix representing this ninety-degree rotation about the x -axis. The grasp pose $\mathbf{T}_g$ is subsequently defined as

$$\mathbf{T}_g^0 = \begin{bmatrix} R_x(\pi) & 0 \\ 0 & 1 \end{bmatrix} \cdot \mathbf{T}_{wt}, R_x(\pi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} \quad (16)$$

2.3.2. Grasp Pose Optimization

The ellipsoid model and initial grasp pose were established, but a PCA-based model is insufficient for complex environments. Our algorithm adjusts the pitch angle based on the environment and target distance, expanding the workspace and enhancing robustness. Pitch and yaw angles are analyzed separately.

The pitch angle of the gripper is influenced by two factors: the target's height relative to the robot. For a given grasping target, its height relative to the robot arm's base can be represented as h , and the distance of the target from the robot arm's base in the XOY plane can be expressed as $d = \sqrt{x^2 + y^2}$. The parameters α and β are set based on the operational range of the robot arm. The graphical representation of this function is shown in Figure 6.

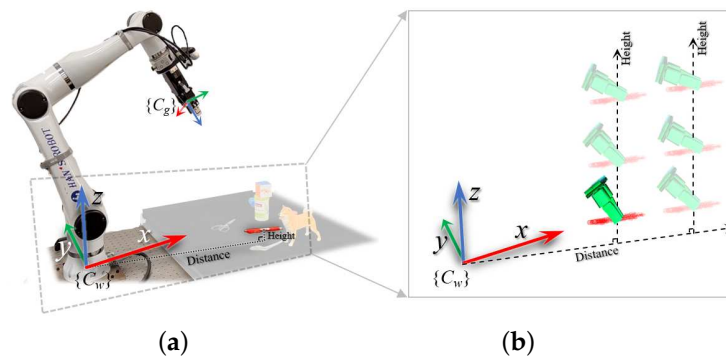


Figure 6. Optimization of grasp pose: (a) influence of target distance and height on pitch angle; (b) grasp adjustments for varying heights and distances.

In robotic grasping systems, the pitch angle of the grasping pose critically affects the success rate and stability. To optimize grasping performance, we propose a Gaussian-based pitch angle modeling method that accounts for variations in the horizontal distance and vertical height between the target and the robotic arm's base.

First, as the horizontal distance d between the target and the robot arm's base increases, the pitch angle θ should gradually decrease, allowing the robotic arm to extend further and cover a larger workspace. Similarly, as the vertical height h of the target increases, the pitch angle θ should also decrease appropriately to ensure stable grasping over a wider range of heights. Based on this design concept, we model the pitch angle θ as a two-dimensional Gaussian function of the horizontal distance d and vertical height h :

$$\theta(h, d) = \theta_0 \cdot \exp\left(-\frac{(d - d_0)^2}{2\sigma_d^2} - \frac{(h - h_0)^2}{2\sigma_h^2}\right) \quad (17)$$

where θ_0 is the maximum value of the pitch angle, and d_0 and h_0 represent the horizontal distance and vertical height at which the pitch angle reaches its maximum value, respectively. The parameters σ_d and σ_h control the spread of the Gaussian distribution along

the horizontal distance and vertical height, determining the rate at which the pitch angle decays as d and h change.

After obtaining the optimal pitch angle θ^* , the grasping pose is fine-tuned, and the transformation matrix is given by

$$R_x(\theta^*) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta^* & -\sin \theta^* \\ 0 & \sin \theta^* & \cos \theta^* \end{bmatrix} \quad (18)$$

With this modeling approach, the pitch angle reaches its maximum value when the target is near $d = d_0$ and $h = h_0$, and it gradually decreases following a Gaussian distribution as the target deviates in horizontal distance or vertical height. This approach integrates target position with grasp pose, maximizing workspace and enhancing flexibility. Figure 6b illustrates grasp adjustments and optimizations for different heights and distances.

2.3.3. Obstacle Avoidance Design

This section builds upon the previous discussion, where we modeled the target and established the initial grasp pose, and optimized the pitch angle. Now, we focus on adjusting the yaw angle to improve obstacle avoidance in complex environments.

Figure 7 illustrates various grasping scenarios: (a) grasping a pen on a flat surface, (b) grasping a target on an inclined surface requiring yaw angle adjustment, and (c) grasping a target in a cluttered environment that demands flexible yaw optimization to avoid obstacles. This approach aims to enhance obstacle avoidance by adaptively adjusting the yaw angle based on the surrounding environment.

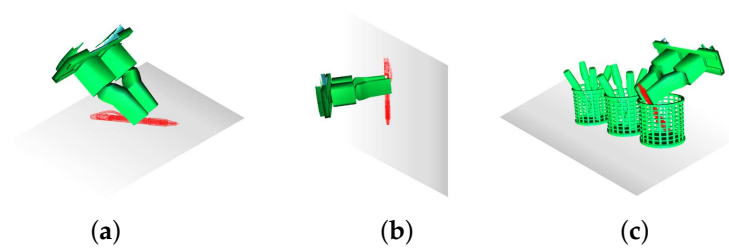


Figure 7. Grasp pose adjustments in complex environments: (a) grasping a pen on a flat surface; (b) grasping a target on an inclined surface; (c) grasping a target in a cluttered environment.

To optimize the grasp pose, we define the grasp pose as $\mathbf{T}_g$, where the approach direction aligns with the z -axis, and the gripper's opening direction is along the x -axis. Given the point cloud of the target object P_t and the neighborhood point cloud P_r within a radius r around the object, we denote the yaw angle of the robotic arm as α .

To assess the potential for collisions between the robotic arm and obstacles during the grasping process, we introduce an optimization objective function based on the variance of the projection distribution of the neighborhood point cloud P_r along the z -axis:

$$J(\alpha) = \text{Var}(\{z_i(\alpha) | \forall \mathbf{p}_i \in P_r\}) \quad (19)$$

Here, $\mathbf{p}_i = (x_i, y_i, z_i)$ represents any point in the neighborhood point cloud P_r , and $z_i(\alpha)$ denotes the projection of point P_i on the z -axis under the yaw angle α .

To make this variance more intuitive, the objective function is rewritten as

$$J(\alpha) = \frac{1}{|P_r|} \sum_{i=1}^{|P_r|} (z_i(\alpha) - \bar{z}(\alpha))^2 \quad (20)$$

where $|P_r|$ is the number of points in the neighborhood point cloud, and $\bar{z}(\alpha)$ is the mean projection of the neighborhood point cloud on the z-axis under the yaw angle α , calculated as follows:

$$\bar{z}(\alpha) = \frac{1}{|P_r|} \sum_{i=1}^{|P_r|} z_i(\alpha) \quad (21)$$

By minimizing the objective function $J(\alpha)$, the projection of the neighborhood point cloud on the z-axis can be made more concentrated, thereby reducing the likelihood of collisions between the robotic arm and obstacles during the grasping process.

This optimization method not only enables the grasp pose to better adapt to the current environment but also significantly enhances the stability and success rate of the grasp in practical applications. The optimization process can be carried out using methods such as gradient descent, genetic algorithms, or simulated annealing, ultimately yielding the optimal yaw angle α^* to guide the actual grasping operation. In summary, the implementation details of our algorithm are shown in the pseudocode of Algorithm 1.

Algorithm 1: Grasp pose estimation and optimization.

Input: $\mathbb{P}_t$: target point cloud, $\mathbb{P}_r$: surrounding point cloud, $\mathbf{T}_g$: initial grasp pose;
 α : initial yaw angle; h : target height; d : horizontal distance

Output: $\mathbf{T}_g^*$: optimized grasp pose

- 1 **Step1 : EllipsoidModeling ;**
 - 2 Construct matrix A and vector b from target point cloud $\mathbb{P}_t$; ▷ Equation (6)
 - 3 Solve $A \cdot X = b$ using least squares;
 - 4 Extract center parameters (x_0, y_0, z_0) from X ;
 - 5 Construct quadratic matrix Q from A ;
 - 6 Perform eigenvalue decomposition on Q : $Q = R \cdot \Lambda \cdot R^T$; ▷ Equation (10)
 - 7 Compute semi-axes (a, b, c) from eigenvalues $\lambda_1, \lambda_2, \lambda_3$;
 - 8 Construct transformation matrix $\mathbf{T}_{wt}$ and $\mathbf{T}_g$ from R and (x_0, y_0, z_0) ; ▷ Equations (15) and (16)
 - 9 **Step2 : PitchAngleOptimization ;**
 - 10 Calculate pitch angle function $\theta(h, d)$ ▷ Equation (17);
 - 11 Calculate the optimized rotation matrix $R_x(\theta^*)$ ▷ Equation (18);
 - 12 Update the rotation part of $\mathbf{T}_g$: $\mathbf{R}_g^* = R_x(\theta^*) \cdot \mathbf{R}_g$;
 - 13 Update grasp pose $\mathbf{T}_g^* = \begin{bmatrix} \mathbf{R}_g^* & \mathbf{t}_g \\ 0 & 1 \end{bmatrix}$;
 - 14 **Step3 : YawAngleOptimization ;**
 - 15 $\Delta \text{Var}(\alpha) = 10e5, \text{Var}(\alpha^{\text{last}}) = 10e5, \text{iter} = 0, \text{max_iter} = 100$;
 - 16 **while** $\Delta \text{Var}(\alpha) > 10^{-5}$ **and** $\text{iter} < \text{max_iter}$ **do**
 - 17 obtain yaw angle α^{iter} from $\mathbf{T}_g^{\text{curr}}$;
 - 18 **foreach** $p_j \in \mathbb{P}_r$ **do**
 - 19 Compute the projection $z_j(\alpha^{\text{iter}})$ of p_j along the z-axis using the rotation matrix $R(\alpha^{\text{iter}})$ ▷ Equation (20)
 - 20 Compute the mean projection $\bar{z}(\alpha^{\text{iter}})$;
 - 21 Compute the variance of projections $\text{Var}(\alpha^{\text{iter}})$;
 - 22 Calculate the Jacobian matrix $\frac{\partial \text{Var}}{\partial \mathbf{T}}$ by using local linearization in SE(3) space.
 - 23 obtain pose incremental ξ by using LM algorithm;
 - 24 $\Delta \text{Var}(\alpha) = \text{Var}(\alpha^{\text{last}}) - \text{Var}(\alpha^{\text{iter}})$;
 - 25 Update new pose $\mathbf{T}_g^{\text{curr}} \leftarrow \mathbf{T}_g^{\text{curr}} \oplus \xi$; $\text{Var}(\alpha^{\text{last}}) \leftarrow \text{Var}(\alpha^{\text{iter}})$
 - 26 Update $\mathbf{T}_g^*$ with α^* and θ^* ;
 - 27 **return** $\mathbf{T}_g^*$;
-

3. Experiment

The experiments included both simulation and real-world tests on a platform with an Intel Core i9-10700 CPU, NVIDIA GTX 2060Ti GPU, and Ubuntu 20.04.

3.1. Simulation Experiments

Grasp stability and success are influenced by point cloud noise and environmental complexity. Point cloud noise arises from sensor accuracy, environmental factors, and motion dynamics, which can significantly affect the performance of grasping algorithms. In real-world applications, sensor errors, variations in ambient lighting, surface reflectivity, and occlusions introduce various levels of noise, leading to inaccurate depth data or incomplete point clouds. This noise can reduce the accuracy of object recognition, pose estimation, and grasp planning, potentially resulting in grasp failures or collisions. Therefore, the robustness of the algorithm to noise is crucial for improving stability in practical scenarios.

To further assess the impact of point cloud noise on grasping performance, we conducted simulation experiments using Gazebo, a widely used platform for robotic simulation. Gazebo simulations were conducted with the robotic arm operating between 0.5 and 1.5 m, using an RGB-D camera. Gazebo, with ODE dynamics and ROS integration, provides realistic simulations and motion control.

Various common objects with different scales and shapes, including a pen, a banana, a mobile phone, an apple, a soda can, and a game controller, were selected as grasping targets. These objects were randomly placed on a table within the testing environment and tested at three different horizontal distances from the robotic arm's origin, 0.6–0.8 m (d_1), 0.8–1.0 m (d_2), and 1.0–1.2 m (d_3), which were used as the first control variable. The experimental setup is shown in Figure 8.

To evaluate the algorithm's stability under different point cloud noise conditions, random noise following a normal distribution was added to the coordinates of each target point, defined as $\mathbf{p}'_i = \mathbf{p}_i + \mathbf{n}_i$, where $\mathbf{P}_i$ represents the i th point in the original point cloud, $\mathbf{p}'_i$ is the point with added noise, and $\mathbf{n}_i \sim \mathcal{N}(0, \sigma^2)$ denotes Gaussian noise with a mean of 0 and a variance of σ^2 . The noise levels were set at $\sigma = 5$ mm (low noise), $\sigma = 10$ mm (medium noise), and $\sigma = 15$ mm (high noise).

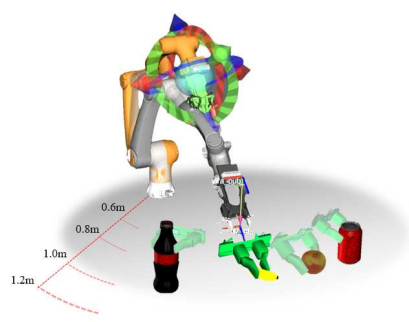


Figure 8. Simulation experiment setup.

Each group was repeated 1000 times in the simulation. A successful grasp required lifting the target over 20 cm with 4 m/s² acceleration. TGSr was calculated and compared to learning-based methods GPD [19] and PointNetGPD [20]. The experimental results are shown in Figure 9, where (a), (b), and (c) represent the TGSr (%) results under point cloud noise levels of $\sigma = 5$ mm, $\sigma = 10$ mm, and $\sigma = 15$ mm, respectively. The results indicate that for medium-distance targets (0.8–1.0 m), PointNetGPD and the proposed method both achieved TGSrs above 70% in low noise, with the proposed method reaching over 83%. Under high noise, GPD and PointNetGPD rates dropped by 15.3% and 8.5%,

respectively, due to reliance on point cloud stability. The proposed method showed only a 4.9% reduction, demonstrating greater robustness.

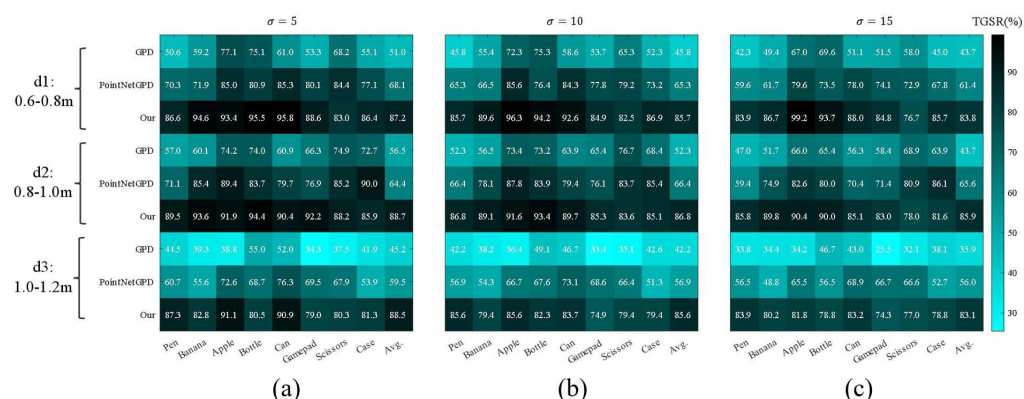


Figure 9. Grasp success rate (TGSr) comparison for different methods under varying noise levels: (a) TGSr at low noise; (b) TGSr at medium noise; (c) TGSr at high noise.

The averaged TGSr results under different noise levels are shown in Table 1. All methods performed best at medium distances, as short and long distances can cause kinematic singularities, leading to failures. The proposed method uses a nonlinear adjustment factor for adapting to different distances, achieving higher average TGSr across all scenarios.

Table 1. Mean TGSr (%) in three noise environments.

Dist.	Method\Target	Pen	Banana	Apple	Bottle	Can	Gamepad	Scissors	Case	Avg.
d1	GPD	46.2	54.7	72.1	73.3	56.9	52.8	63.8	50.8	46.8
	PointNetGPD	65.1	66.7	83.4	76.9	82.6	77.3	78.8	72.7	64.9
	Our Method	85.4	90.3	96.3	94.5	92.1	86.1	80.8	86.4	85.6
d2	GPD	52.1	56.1	71.2	70.9	60.4	63.4	73.5	68.3	50.8
	PointNetGPD	65.6	79.5	86.6	82.6	76.5	74.8	83.3	87.2	65.5
	Our Method	87.4	90.8	91.3	92.6	88.4	86.8	83.2	84.2	87.1
d3	GPD	40.2	37.3	36.5	50.3	47.2	31.1	34.9	40.8	41.1
	PointNetGPD	58.0	52.9	68.3	64.3	72.8	68.3	67.0	52.6	57.5
	Our Method	85.6	80.8	86.2	80.5	85.9	76.1	78.9	79.8	85.8

3.2. Real-World Experiments

To validate the proposed method's performance, two grasping scenarios were configured: static and dynamic. In the static scenario, an Elfin5 robotic arm with an AG_95 gripper (maximum opening width of approximately 12 cm) was used for tabletop grasping, as illustrated in Figure 10. Six objects—box, bottle, glasses case, scissors, pen, and tape—were selected as targets, representing common small to medium-sized desk items (volumes $< 0.5 \text{ dm}^3$).

The dynamic scenario, presenting greater challenges, employed a Robotiq 2F-85 gripper (maximum opening width 20 cm) mounted on a mobile platform for grasping from both the front and back of a long table. Target objects included a bottle, cup, game controller, mouse, and various fruits, representing medium-sized everyday items (volumes $0.2\text{--}1.5 \text{ dm}^3$). Objects were densely arranged to increase the task complexity.



Figure 10. Real-world grasping scenarios: (a) static tabletop grasping scenario; (b) dynamic grasping scenario with mobile platform.

The Realsense-D455 was used for both experiments. The proposed method showed a 56.3% and 48.6% higher efficiency compared to GPD and PointNetGPD, relying on model optimization instead of high-performance GPUs, as shown in Figure 11.

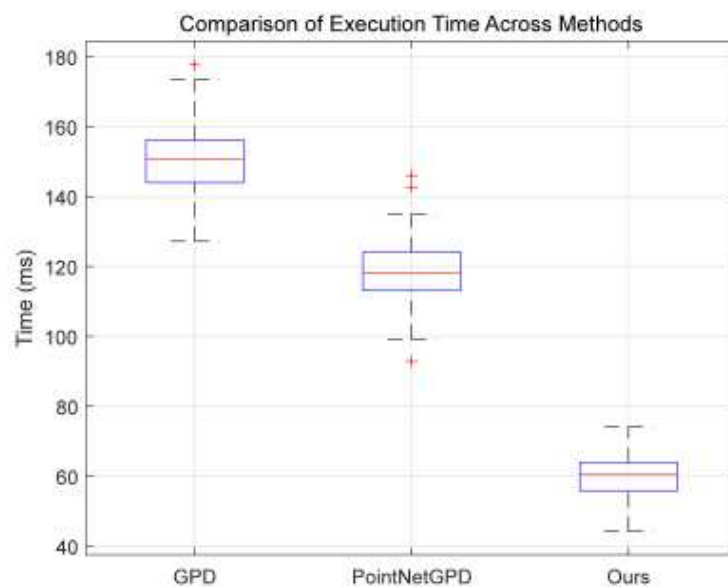


Figure 11. Box Plot comparing grasp pose estimation time for three methods. (The boxplot illustrates the distribution of the data, where the blue box represents the interquartile range (IQR) with the lower edge at the first quartile (Q1), the upper edge at the third quartile (Q3), and the middle line representing the median (Q2). The red line indicates the mean value of the data, while the red cross marks the outliers, which are data points significantly different from the majority of the data, typically located beyond 1.5 times the IQR from the quartiles.)

In the static experiment, all objects were placed on a black mat, and the grasping performance of six objects was tested individually. Each target was repositioned 20 times for grasp attempts, and the success rates were evaluated. Figure 12 and Table 2 show the different grasp postures and test results for grasping a bottle, pen, and scissors.

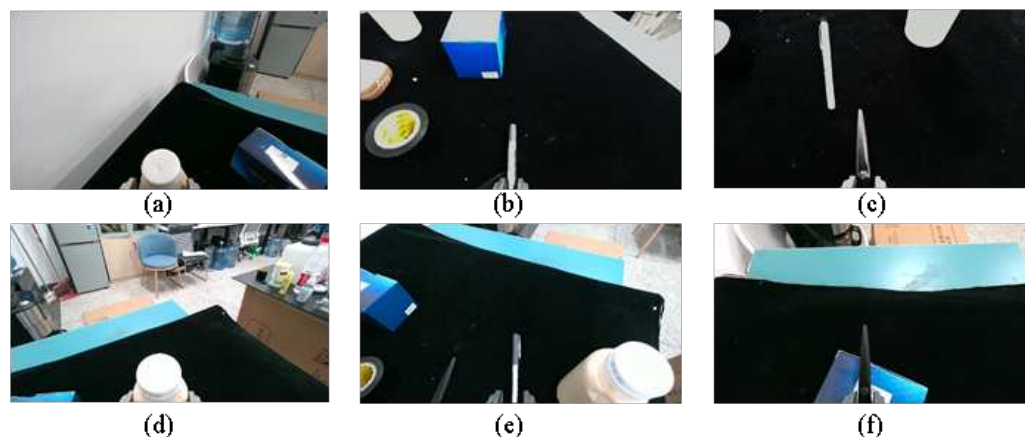


Figure 12. Example of target grasp execution, with images captured by Realsense D455 Color Camera, with the gripper end at the bottom of the image. (Figure (a): Grasping the bottle on the left side of the robot; Figure (b): Grasping the pen; Figure (c): Grasping the scissors; Figure (d): Grasping the bottle on the right side of the robot; Figure (e): Grasping the pen placed directly next to the bottle; Figure (f): Grasping the scissors on the box.)

Table 2. Target grasp success rates (%) of three methods in real-world static experiments.

Target	Box	Bottle	Glass Case	Scissors	Pen	Glue
GPD	50	45	45	50	30	55
PointNetGPD	60	65	75	55	50	75
Proposed Method	95	90	100	95	100	95

The proposed method achieved a 100% grasp success rate for pen and glass case targets in all 20 attempts, demonstrating its stability and robustness. Other methods performed worse due to discrepancies between training and sensor-generated data, leading to incorrect grasps and poor repeatability, revealing a limitation of learning-based approaches.

In dynamic experiments, 20 targets were arranged as shown in Figure 13 and grasped sequentially, with three attempts per target. Experiment 1 grasped from the front, and Experiment 2 from the back. Success rates, calculated as the ratio of successful grasps to total attempts, are shown in Table 3.



Figure 13. Target set in dynamic scenarios.

Table 3. Attempts count and target grasp success rates (%) of three methods in real-world dynamic experiments.

Group	GPD		PointNetGPD		Our Method	
	Attempts Count	Success Rate (%)	Attempts Count	Success Rate (%)	Attempts Count	Success Rate (%)
1	51	50	45	80	27	100
2	47	60	41	75	30	95

The results presented in Table 3 indicate that the proposed method achieves a higher grasp success rate with fewer attempts, which is crucial for real-world robotic applications where precision is challenging.

4. Discussion

The grasp pose estimation algorithm proposed in this study represents a significant advancement in robotic arm manipulation, particularly for systems using consumer-grade RGB-D depth sensors. Extensive experiments conducted in both simulation environments and real-world scenarios demonstrate that this method significantly outperforms existing deep learning-based grasping algorithms, such as GPD and PointNetGPD, in terms of grasp success rate and computational efficiency. Specifically, under varying experimental conditions, the proposed algorithm exhibits stable performance in complex environments, while significantly enhancing computational efficiency and reducing reliance on high-performance computing resources, highlighting its broad applicability in both industrial and consumer robotics. In contrast to traditional deep learning methods, this approach does not rely on large annotated datasets or complex neural network training, resulting in lower computational costs and higher adaptability, enabling it to run on hardware platforms with limited resources.

The success of the algorithm can also be attributed to the application of nonlinear optimization techniques. By fitting ellipsoids to the point cloud, we are able to perform principal component analysis of the target's geometric distribution, effectively describing the shape of the target object, particularly for objects with simple geometric structures. Additionally, this method incorporates the surrounding environment's features into the nonlinear optimization for pose estimation, enabling the robotic arm to perform obstacle-avoiding grasps in the scene. However, despite its strong performance in most scenarios, the ellipsoidal model may not accurately capture the true shape of objects with complex topological structures or in dynamic environments, thereby affecting the grasp success rate. Specifically, when the surface geometry of the target object is irregular, the optimal grasp pose may not be derived solely from geometric distribution analysis, but often requires the integration of empirical constraints. For example, objects such as teapots or pots with handles may necessitate the introduction of neural network analysis at certain stages.

To enhance the stability of the algorithm in diverse environments, future research will focus on further optimizing the model to better address these challenges. Future work will concentrate on several key aspects: First, more precise geometric modeling methods, such as multimodal fusion, adaptive surface modeling, and point cloud completion, will be introduced to further improve the algorithm's adaptability and accuracy. Second, hybrid models based on deep learning will be explored by combining traditional geometric modeling with deep learning approaches, thereby enhancing the algorithm's generalization ability in complex scenarios.

5. Conclusions

In this study, the proposed grasp pose estimation algorithm demonstrated strong robustness and efficiency across multiple experiments. We simulated the impact of different sensor accuracies and environmental conditions on grasp performance by using varying levels of noise (low, medium, and high noise). Under low-noise conditions, the algorithm was able to estimate object poses with high accuracy, achieving a success rate close to 90%. As noise levels increased, the grasp success rates of other comparison methods dropped significantly, highlighting the non-negligible impact of noise on the algorithm's performance. Under medium-noise conditions, the proposed algorithm achieved a grasp success rate (TGSR) exceeding 87%, outperforming GPD and PointNetGPD by 20% and 10%, respec-

tively. This method effectively simplifies the modeling of object geometry, and the strategy of incorporating environmental constraints into pose optimization enables the robot to maintain high performance in cluttered environments. The algorithm also demonstrated good adaptability when handling objects of varying shapes and sizes, maintaining stable performance across different test objects.

Author Contributions: Conceptualization, A.C., X.L., K.C. and C.H.; methodology, A.C., X.L. and K.C.; software, A.C., X.L. and K.C.; validation, A.C. and C.H.; formal analysis, A.C., X.L. and K.C.; investigation, A.C., X.L. and K.C.; resources, A.C., X.L. and K.C.; data curation, A.C., X.L. and K.C.; writing—original draft preparation, A.C., X.L. and K.C.; writing—review and editing, A.C., X.L., K.C. and C.H.; visualization, A.C., X.L. and K.C.; funding acquisition, A.C. All authors have contributed to this work and have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Science and Technology Development Fund of Macau SAR, grant number FDCT 005/2022/ALC.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Data underlying the results presented in this paper are not publicly available at this time but may be obtained from the authors upon reasonable request.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Belanche, D.; Casaló, L.V.; Flavián, C.; Schepers, J. Service robot implementation: A theoretical framework and research agenda. *Serv. Ind. J.* **2020**, *40*, 203–225. [[CrossRef](#)]
2. Varley, J.; Weisz, J.; Weiss, J.; Allen, P. Generating multi-fingered robotic grasps via deep learning. In Proceedings of the 2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Hamburg, Germany, 28 September–2 October 2015; IEEE: Piscataway, NJ, USA, 2015; pp. 4524–4530.
3. Fang, H.-S.; Wang, C.; Fang, H.; Gou, M.; Liu, J.; Yan, H.; Liu, W.; Xie, Y.; Lu, C. Anygrasp: Robust and efficient grasp perception in spatial and temporal domains. *IEEE Trans. Robot.* **2023**, *5*, 3929–3945. [[CrossRef](#)]
4. Mahler, J.; Pokorný, F.T.; Hou, B.; Roderick, M.; Laskey, M.; Aubry, M.; Kohlhoff, K.; Kröger, T.; Kuffner, J.; Goldberg, K. Dex-net 1.0: A cloud-based network of 3D objects for robust grasp planning using a multi-armed bandit model with correlated rewards. In Proceedings of the 2016 IEEE International Conference on Robotics and Automation (ICRA), Stockholm, Sweden, 16–21 May 2016; IEEE: Piscataway, NJ, USA, 2016; pp. 1954–1961.
5. Zhang, Y.; Adin, V.; Bader, S.; Oelmann, B. Leveraging Acoustic Emission and Machine Learning for Concrete Materials Damage Classification on Embedded Devices. *IEEE Trans. Instrum. Meas.* **2023**, *72*, 2525108. [[CrossRef](#)]
6. Lenz, I.; Lee, H.; Saxena, A. Deep learning for detecting robotic grasps. *Int. J. Robot. Res.* **2015**, *34*, 705–724. [[CrossRef](#)]
7. Xie, Z.; Liang, X.; Roberto, C.J. Learning-based robotic grasping: A review. *Front. Robot. AI* **2023**, *10*, 1038658. [[CrossRef](#)] [[PubMed](#)]
8. Sundermeyer, M.; Mousavian, A.; Triebel, R.; Fox, D. Contact-graspnet: Efficient 6-DOF grasp generation in cluttered scenes. In Proceedings of the 2021 IEEE International Conference on Robotics and Automation (ICRA), Xi'an, China, 30 May–5 June 2021; IEEE: Piscataway, NJ, USA, 2021; pp. 4244–4250.
9. Bicchi, A.; Kumar, V. Robotic grasping and contact: A review. In Proceedings of the 2000 ICRA Millennium Conference IEEE International Conference on Robotics and Automation Symposia Proceedings, San Francisco, CA, USA, 24–28 April 2000; IEEE: Piscataway, NJ, USA, 2000; pp. 348–353.
10. Boubekri, N.; Chakraborty, P.J. Robotic grasping: Gripper designs, control methods and grasp configurations—A review of research. *Int. J. Mech. Sci.* **2002**, *13*, 520–531. [[CrossRef](#)]
11. Joshi, S.; Kumra, S.; Sahin, F. Robotic grasping using deep reinforcement learning. In Proceedings of the 2020 IEEE 16th International Conference on Automation Science and Engineering (CASE), Washington, DC, USA, 22–26 August 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 351–357.
12. Guo, N.; Zhang, B.; Zhou, J.; Zhan, K.; Lai, S. Pose estimation and adaptable grasp configuration with point cloud registration and geometry understanding for fruit grasp planning. *J. Comput. Agric. Eng.* **2020**, *179*, 105818. [[CrossRef](#)]

13. Ni, P.; Zhang, W.; Zhu, X.; Cao, Q. Pointnet++ grasping: Learning an end-to-end spatial grasp generation algorithm from sparse point clouds. In Proceedings of the 2020 IEEE International Conference on Robotics and Automation (ICRA), Paris, France, 31 May–4 June 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 4597–4603.
14. Qian, K.; Jing, X.; Duan, Y.; Zhou, B.; Fang, F.; Xia, J.; Ma, X. Grasp pose detection with affordance-based task constraint learning in single-view point clouds. *J. Robot. Autom.* **2020**, *100*, 145–163. [\[CrossRef\]](#)
15. Qi, C.R.; Su, H.; Mo, K.; Guibas, L.J. Pointnet: Deep learning on point sets for 3D classification and segmentation. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Honolulu, HI, USA, 21–26 July 2017; IEEE: Piscataway, NJ, USA, 2017; pp. 77–85.
16. Krizhevsky, A.; Sutskever, I.; Hinton, G.E. Imagenet classification with deep convolutional neural networks. In Proceedings of the Advances in Neural Information Processing Systems 25 (NIPS 2012), Lake Tahoe, NV, USA, 3–6 December 2012; Volume 25.
17. Zhou, L.; Sun, G.; Li, Y.; Li, W.; Su, Z.J. Point cloud denoising review: From classical to deep learning-based approaches. *J. Geom. Model.* **2022**, *121*, 101140. [\[CrossRef\]](#)
18. Nguyen, V.-D. Constructing force-closure grasp. *J. Robot. Res.* **1988**, *7*, 157–171. [\[CrossRef\]](#)
19. Pas, A.; Gualtieri, M.; Saenko, K.; Platt, R. Grasp pose detection in point clouds. *Int. J. Robot. Res.* **2017**, *36*, 1455–1473.
20. Liang, H.; Ma, X.; Li, S.; Görner, M.; Tang, S.; Fang, B.; Sun, F.; Zhang, J. Pointnetgpd: Detecting grasp configurations from point sets. In Proceedings of the 2019 IEEE International Conference on Robotics and Automation (ICRA), Montreal, QC, Canada, 20–24 May 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 1954–1961.
21. Dharbaneshwer, S.; Subramanian, S.J.; Kohlhoff, K.J.M. Robotic grasp analysis using deformable solid mechanics. *J. Mech. Eng.* **2019**, *54*, 1767–1784. [\[CrossRef\]](#)
22. Pierson, H.A.; Gashler, M.S. Deep learning in robotics: A review of recent research. *J. Robot. Res.* **2017**, *31*, 821–835. [\[CrossRef\]](#)
23. Billard, A.; Kragic, D. Trends and challenges in robot manipulation. *Science* **2019**, *364*, eaat8414. [\[CrossRef\]](#) [\[PubMed\]](#)
24. Caldera, S.; Rassau, A.; Chai, D. Review of deep learning methods in robotic grasp detection. *J. Mach. Interact.* **2018**, *2*, 57. [\[CrossRef\]](#)
25. Mohammed, M.Q.; Chung, K.L.; Chyi, C.S. Review of deep reinforcement learning-based object grasping: Techniques, open challenges, and recommendations. *J. Intell. Autom.* **2020**, *8*, 178450–178481. [\[CrossRef\]](#)
26. Hinterstoisser, S.; Holzer, S.; Cagniart, C.; Ilic, S.; Konolige, K.; Navab, N.; Lepetit, V. Multimodal templates for real-time detection of texture-less objects in heavily cluttered scenes. In Proceedings of the 2011 International Conference on Computer Vision (ICCV), Barcelona, Spain, 6–13 November 2011; IEEE: Piscataway, NJ, USA, 2011; pp. 858–865.
27. Dong, M.; Zhang, J. A review of robotic grasp detection technology. *J. Robot. Artif. Intell.* **2023**, *41*, 3846–3885. [\[CrossRef\]](#)
28. Hannah, R.; Aron, A.R. Towards real-world generalizability of a circuit for action-stopping. *J. Neurosci. Res.* **2021**, *22*, 538–552. [\[CrossRef\]](#) [\[PubMed\]](#)
29. Zhang, C.; Han, D.; Qiao, Y.; Kim, J.U.; Bae, S.-H.; Lee, S.; Hong, C.S. Faster segment anything: Towards lightweight SAM for mobile applications. *J. Comput. Vis.* **2023**, 1–12.
30. Takaya, K.; Asai, T.; Kroumov, V.; Smarandache, F. Simulation environment for mobile robots testing using ROS and Gazebo. In Proceedings of the 2016 20th International Conference on System Theory, Control and Computing (ICSTCC), Sinaia, Romania, 19–21 October 2016; IEEE: Piscataway, NJ, USA, 2016; pp. 157–162.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.